

课程笔记

CS224N Lecture 1: Intro and Word Vectors

整理：AI Course Notes Contributors；授课：Chris Manning

2024 年 4 月

课程版本：2024 年中文讲义整理版

笔记来源：AI Course Notes Contributors

本版处理：移除课件截图和对应图注，不包含 Stanford 原始课件、图片或视频。

授权：笔记依 CC BY-NC-SA 4.0 分享；完整许可与改动记录见下一页。

来源与许可

本中文笔记由 AI Course Notes Contributors 整理，课程版本为 2024；源稿：<https://github.com/hqhq1025/ai-course-notes/tree/717e2df65c3d2eee593c171156a30d4f8f6812b9/cs224n/lecture01>。

笔记内容依 CC BY-NC-SA 4.0 分享：<https://creativecommons.org/licenses/by-nc-sa/4.0/>。

本副本的改动：移除 Stanford 原始课件截图与依赖这些截图的图注，移除课程视频封面/链接，并清理 TeX 源稿中的行尾空白后重新编译为可独立阅读的 PDF；同时加入本来源与许可说明。完整许可文本随本文件夹提供。Stanford 课件、课件图片和视频不包含在该授权内；请从对应年份 Stanford 课程页访问原始课件。

目录

1 引言：课程概览与学习目标	3
1.1 课程学习目标	3
1.2 课程结构与作业安排	3
1.3 本章小结	3
2 人类语言与词义表示	3
2.1 语言：人类智能的核心驱动力	3
2.2 深度学习驱动的 NLP 革命	4
2.2.1 机器翻译	4
2.2.2 从搜索引擎到问答引擎	4
2.2.3 GPT-2 与大语言模型的崛起	5
2.2.4 ChatGPT 与指令遵循	5
2.2.5 多模态基础模型	5
2.3 本章小结	5
3 从离散符号到分布式表示	5
3.1 传统词义表示的困境	5
3.1.1 WordNet 的局限	5
3.1.2 One-hot 向量的问题	6
3.2 分布式语义与词向量	6
3.2.1 分布式假说	6
3.2.2 词向量 (Word Vectors)	6
3.2.3 词向量空间的可视化	7
3.3 本章小结	7
4 Word2Vec 算法详解	7
4.1 Word2Vec 的核心思想	7
4.2 似然函数与目标函数	8
4.2.1 似然函数	8
4.2.2 从似然到损失函数	8
4.3 Softmax 概率计算	9
4.3.1 两套向量的设计	9
4.3.2 Softmax 公式	9
4.4 模型参数与优化	9
4.4.1 参数空间	9
4.4.2 梯度下降优化	10
4.5 梯度推导	10
4.5.1 分解对数概率	10
4.5.2 项 1 的求导	10
4.5.3 项 2 的求导 (链式法则)	11
4.5.4 合并得到最终结果	11
4.6 随机梯度下降 (SGD)	12

4.7	本章小结	12
5	总结与延伸	12
5.1	讲者的核心总结	12
5.2	全课知识图谱	13
5.3	关键 Takeaways	13
5.4	拓展阅读	13

1 引言：课程概览与学习目标

CS224N (Natural Language Processing with Deep Learning) 是 Stanford 大学最受欢迎的 NLP 课程之一，由 Christopher Manning 教授主讲。本节课作为第一讲，涵盖三大主题：(1) 课程介绍与学习目标；(2) 人类语言与词义表示；(3) Word2Vec 算法详解。

1.1 课程学习目标

Manning 教授明确列出了三个核心学习目标：

CS224N 三大学习目标

1. **掌握深度学习在 NLP 中的基础与前沿方法**：从词向量、前馈神经网络、循环网络、注意力机制，到 Transformer、编码器-解码器模型、大语言模型的预训练与后训练、模型适配、可解释性、智能体等
2. **理解人类语言的特性与计算挑战**：了解语言结构的复杂性，以及为什么让计算机理解和生成人类语言如此困难
3. **具备构建实际 NLP 系统的能力**：不仅学习理论，还能在实际工作中独立搭建文本分类、信息抽取、机器翻译等系统

1.2 课程结构与作业安排

课程包含四个编程作业和一个期末项目：

- **Assignment 1**：入门作业 (Jupyter Notebook)，探索词向量
- **Assignment 2**：数学推导 + PyTorch 入门，构建依存句法分析器
- **Assignment 3-4**：使用 PyTorch + GPU 完成机器翻译和 Transformer 相关项目
- **期末项目**：默认项目（有脚手架）或自选项目（1-3 人组队）

1.3 本章小结

CS224N 是一门从底层到前沿的 NLP 课程，强调理论与实践并重。课程从词向量出发，逐步深入到 Transformer、大语言模型等前沿技术，最终要求学生具备独立构建 NLP 系统的能力。

2 人类语言与词义表示

2.1 语言：人类智能的核心驱动力

Manning 教授从语言学家的视角出发，论证了语言在人类智能中的核心地位。他将人类与近亲黑猩猩进行了对比：

语言与人类文明

黑猩猩在很多方面与人类相似——它们会使用工具、能规划解决问题、甚至短期记忆优于人类。但人类与黑猩猩之间最关键的区别就是**语言**。语言不仅是交流工具，还是高层次思维的支架（scaffolding），使人类能够进行更精细的推理和规划。

语言在人类文明中扮演了三重角色：

1. **交流工具**：使人类能够协作，实现“人类主宰”（human ascendancy）
2. **思维支架**：为高层次认知提供结构化框架——我们用语言来思考、计划、推理
3. **知识载体**：书写系统（约 5000 年历史）使知识能够跨越时间和空间传播，从青铜时代到现代科技

Manning 还引用了 Stanford 心理学家 Herb Clark 的名言：

语言的本质

“The common misconception is that language use has primarily to do with words and what they mean. It doesn't. It has primarily to do with people and what they mean.” ——Herb Clark

语言使用的核心不在于词语及其含义，而在于人以及人想要表达的意思。

语言不是静态的

语言不是从天而降的固定系统——它由人类构造，并随每一代人不断变化。大部分语言创新发生在青少年和年轻人群体中。这意味着任何基于固定规则的 NLP 系统都面临过时的风险。

2.2 深度学习驱动的 NLP 革命

过去十年，深度学习为 NLP 带来了革命性进步。Manning 列举了几个标志性成就：

2.2.1 机器翻译

神经机器翻译（Neural Machine Translation）从 2014 年开始发展，到 2016 年已被 Google 等服务大规模部署。这使得跨语言交流变得前所未有的容易。

2.2.2 从搜索引擎到问答引擎

传统搜索引擎基于关键词匹配返回文档列表。现代系统使用三层神经网络流水线实现真正的**问答**：

1. **检索网络**（Retrieval）：找到与查询语义相似的段落
2. **重排序网络**（Reranking）：对候选段落重新排序
3. **阅读网络**（Reading）：从候选段落中综合信息并生成答案

2.2.3 GPT-2 与大语言模型的崛起

2019 年的 GPT-2 首次展示了大语言模型生成流畅文本的能力。Manning 展示了一个经典示例：给定一个关于“辛辛那提被盗核材料”的开头，GPT-2 能生成连贯的新闻续写，不仅语法正确，还展现出丰富的世界知识——它知道辛辛那提在俄亥俄州、核材料由美国能源部监管等。

GPT-2 的工作原理

GPT-2 的核心机制极其简单：给定所有已有文本，预测下一个最可能出现的词（next word prediction）。然后将预测的词加入已有文本，继续预测下一个词。这种简单的自回归生成方式，在足够大的模型和数据上，产生了惊人的效果。

2.2.4 ChatGPT 与指令遵循

ChatGPT（及 GPT-4）的巨大突破在于**指令遵循**（instruction following）：用户可以用自然语言给出指令，模型能理解并执行。Manning 展示了让 ChatGPT 代写请假邮件的例子——即使输入中包含拼写错误（“song”而非“son”），模型也能正确理解意图。

2.2.5 多模态基础模型

Stanford 提出的**基础模型**（Foundation Models）概念，将大语言模型的技术范式推广到图像、声音、生物信息学等多种模态。Manning 用 DALL-E 生成“火车通过金门大桥”的图片作为示例，展示了文本到图像生成的能力。

2.3 本章小结

人类语言是文明的核心驱动力，同时也是 AI 面临的最具挑战性的问题之一。深度学习在过去十年带来了 NLP 的革命性进步——从机器翻译到 GPT 系列大语言模型，再到多模态基础模型。这些进步的基础，正是本课程将要系统学习的技术。

3 从离散符号到分布式表示

3.1 传统词义表示的困境

在语言学中，词义通常被理解为**指称语义**（denotational semantics）：一个符号（signifier）与一个概念或事物（signified）之间的配对关系。例如，“tree”一词的含义就是世界上所有的树。

然而，在计算机中表示词义，传统方法面临严重困难。

3.1.1 WordNet 的局限

在前神经网络时代，最常用的词义资源是 **WordNet**——一种手工构建的词汇关系网络，记录了同义词、上下位关系等。

WordNet 的三大问题

- **缺乏细微差别**: WordNet 认为 “proficient” 是 “good” 的同义词, 但 “That was a good shot” 不能替换为 “That was a proficient shot”
- **严重不完整**: 缺少现代词汇和俚语 (如 wicked、badass、nifty 等)
- **人工维护成本高**: 需要持续的人力投入来更新和扩展

3.1.2 One-hot 向量的问题

在传统计算系统中, 词被表示为 **one-hot 向量**: 每个词在词表中有一个唯一位置, 对应向量中只有该位置为 1, 其余为 0。

One-hot 表示的根本缺陷

One-hot 向量是**局部表示** (localist representation): 每个词仅在向量的一个位置上被表示。这意味着:

- 任意两个不同词的 one-hot 向量都是**正交**的 (点积 = 0)
- 向量中不包含任何关于词义**相似性**的信息
- 无法从表示本身判断 “motel” 和 “hotel” 比 “motel” 和 “chair” 更相似

解决方案: 学习将相似性编码到向量本身中。

3.2 分布式语义与词向量

3.2.1 分布式假说

一种完全不同的语义观为这个问题提供了解决思路——**分布式语义** (distributional semantics):

分布式假说的哲学渊源

J.R. Firth (英国语言学家, 1957): “You shall know a word by the company it keeps.” (从一个词的上下文可以了解这个词。)

这一思想可追溯到 **Wittgenstein** 的语言哲学: 词的含义在于其**使用** (use), 而非抽象的指称。

分布式语义的核心思想是: 一个词的含义由它经常出现的**上下文** (surrounding words) 决定。如果两个词经常出现在相似的上下文中, 它们就有相似的含义。

3.2.2 词向量 (Word Vectors)

基于分布式语义的思想, 我们可以为每个词学习一个**稠密向量** (dense vector), 使得语义相近的词具有相似的向量表示。

词向量的核心特征

- **稠密**：每个维度都有非零值（区别于 one-hot 的稀疏表示）
- **低维**：典型维度为 100–2000（远小于词表大小，如 500,000）
- **分布式**：词义分布在整个向量的所有维度上（区别于 one-hot 的局部表示）
- **相似性可计算**：语义相似的词具有较大的点积（dot product）

词向量也被称为 **word embeddings**（词嵌入）或 **neural word representations**（神经词表示）。

3.2.3 词向量空间的可视化

将词向量投影到二维空间（通常使用 t-SNE），可以看到语义相近的词聚集在一起：

高维空间的“神奇”特性

Manning 特别强调：高维空间的行为与低维空间**截然不同**。在二维空间中，两个点只有当 x 和 y 坐标都相似时才“接近”。但在高维空间中，一个点可以**同时**在不同维度上靠近不同的点。这就是为什么即使每个词只有一个向量，“star”也能同时靠近天文词汇（nebula、galaxy）和娱乐词汇（celebrity、fame）——这些相似性体现在不同的维度组合上。

一词一向量的局限

在 Word2Vec 中，每个词串（string）只有一个向量。这意味着多义词（如 “bank”：银行/河岸，“star”：恒星/明星）的不同含义被**平均**到一个向量中。后续课程将介绍**上下文词表示**（contextual word representations），如 ELMo、BERT 等，可以根据上下文给出不同的向量。

3.3 本章小结

从 WordNet 的手工规则到 one-hot 的符号表示，传统方法都无法有效捕捉词义的细微差别和相似性。分布式语义假说提供了一条全新的路径：通过词的上下文来学习其含义。词向量将每个词表示为稠密的低维向量，使得语义相似性可以直接通过向量运算来度量。

4 Word2Vec 算法详解

4.1 Word2Vec 的核心思想

Word2Vec 是 Tomas Mikolov 等人于 2013 年在 Google 提出的词向量学习方法。虽然不是第一个词向量算法（词向量的研究可追溯到 2000 年左右），但因其**简洁高效**而广受欢迎。

Word2Vec（Skip-gram 模型）的基本思想：

Word2Vec Skip-gram 模型

1. 准备一个大型文本语料库 (corpus)
2. 为词表中每个词分配一个随机初始化的向量
3. 遍历语料库中的每个位置 t ，取当前位置的词为**中心词** (center word)，取固定窗口 m 内的词为**上下文词** (context/outside words)
4. 利用中心词和上下文词的向量相似度，计算上下文词的出现概率
5. 不断调整所有词向量，使得观察到的上下文的概率尽可能大

语料库 (Corpus)

Corpus 是拉丁语 “body” (身体) 的意思，在 NLP 中指一个大型文本集合。注意其复数形式是 **corpora** (不是 “corpuses” 或 “corpi”)。Manning 在课上特别提醒：如果在作业中写了错误的复数形式，说明没有认真听第一节课！

4.2 似然函数与目标函数

4.2.1 似然函数

对于语料库中的每个位置 t ($t = 1, \dots, T$)，给定中心词 w_t ，我们希望在窗口大小为 m 的范围内，上下文词 w_{t+j} ($-m \leq j \leq m, j \neq 0$) 出现的概率尽可能高。整体似然函数为：

$$L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

- T ：语料库中词的总数
- m ：上下文窗口大小 (Manning 的示例中 $m = 2$)
- θ ：模型的所有参数 (即所有词向量)

4.2.2 从似然到损失函数

将似然函数转化为可优化的损失函数需要两步变换：

1. **取负号**：将最大化问题转为最小化问题 (对应梯度下降)
2. **取对数**：将乘积转为求和，简化数学处理

最终得到**平均负对数似然** (average negative log-likelihood) 作为目标函数：

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

为什么取对数？

- **数学便利**: $\log$ 将乘积变为求和，简化求导过程
- **数值稳定**: 大量小概率相乘会导致数值下溢，取对数后变为负数的求和
- **单调性**: $\log$ 是单调递增函数，最大化 $\log L$ 等价于最大化 L

4.3 Softmax 概率计算

4.3.1 两套向量的设计

Word2Vec 的一个关键设计是：每个词拥有**两个**向量：

- v_w : 当 w 作为**中心词** (center word) 时使用
- u_w : 当 w 作为**上下文词** (context/outside word) 时使用

这一设计使得数学推导更简洁。

4.3.2 Softmax 公式

给定中心词 c 和上下文词 o ，条件概率定义为：

$$P(o | c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

- $u_o^T v_c$: 上下文词向量与中心词向量的**点积**，衡量二者的相似度
- $\exp(\cdot)$: 指数函数，将任意实数映射为正数
- $\sum_{w \in V}$: 对词表中所有词求和，起**归一化**作用，确保概率之和为 1

Softmax 函数的直觉理解

Softmax 是深度学习中将任意实数向量转化为概率分布的标准工具：

1. **点积** $u_o^T v_c$: 度量两个词向量的相似度（无界实数）
2. **取指数** $\exp(\cdot)$: 将实数映射为正数（确保概率非负）
3. **归一化**（除以总和）: 确保所有概率之和为 1

名字中的“soft”表示它是“max”的平滑版本：它**放大了**最大项的概率，但仍然给较小项分配非零概率。

4.4 模型参数与优化

4.4.1 参数空间

Word2Vec 的**全部**参数就是所有词的向量。假设词表大小为 V ，词向量维度为 d ，则：

$$\theta = \begin{bmatrix} v_{\text{aardvark}} \\ v_a \\ \vdots \\ v_{\text{zebra}} \\ u_{\text{aardvark}} \\ u_a \\ \vdots \\ u_{\text{zebra}} \end{bmatrix} \in \mathbb{R}^{2dV}$$

例如，当 $V = 400,000$ 、 $d = 100$ 时，总参数数量为 $2 \times 400,000 \times 100 = 80,000,000$ （8 千万）。

4.4.2 梯度下降优化

优化过程的核心是**梯度下降**（gradient descent）：

1. 随机初始化所有词向量
2. 计算目标函数 $J(\theta)$ 关于所有参数的梯度 $\nabla_{\theta} J(\theta)$
3. 沿梯度的反方向更新参数： $\theta \leftarrow \theta - \alpha \nabla_{\theta} J(\theta)$
4. 重复步骤 2-3 直到收敛

这个过程类似于在高维空间中“走下坡路”：梯度指出了当前位置上升最快的方向，我们朝反方向走一小步，就能降低损失。

4.5 梯度推导

Manning 在课上详细推导了目标函数关于中心词向量 v_c 的梯度。这是理解 Word2Vec 训练过程的关键。

4.5.1 分解对数概率

首先，我们需要计算单个 log 概率项的梯度：

$$\frac{\partial}{\partial v_c} \log P(o | c) = \frac{\partial}{\partial v_c} \log \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

利用对数的性质 $\log \frac{a}{b} = \log a - \log b$ ，将其分解为两项：

$$= \underbrace{\frac{\partial}{\partial v_c} \log \exp(u_o^T v_c)}_{\text{项 1: 分子}} - \underbrace{\frac{\partial}{\partial v_c} \log \sum_{w \in V} \exp(u_w^T v_c)}_{\text{项 2: 分母}}$$

4.5.2 项 1 的求导

分子部分非常简单，log 和 exp 互为逆运算，直接消掉：

$$\frac{\partial}{\partial v_c} \log \exp(u_o^T v_c) = \frac{\partial}{\partial v_c} (u_o^T v_c) = u_o$$

向量点积的导数

对于向量点积 $u_o^T v_c = \sum_{i=1}^d (u_o)_i (v_c)_i$ ，关于 v_c 的偏导数为：

$$\frac{\partial}{\partial (v_c)_j} \sum_{i=1}^d (u_o)_i (v_c)_i = (u_o)_j$$

因为只有 $i = j$ 的项对 $(v_c)_j$ 有贡献。将所有分量拼起来，得到向量 u_o 。
这不是高中单变量微积分——这里的导数是一个向量！

4.5.3 项 2 的求导（链式法则）

分母部分需要两次应用**链式法则**（chain rule）：

第一次链式法则（log 的导数 × 内部函数的导数）：

$$\frac{\partial}{\partial v_c} \log \sum_w \exp(u_w^T v_c) = \frac{1}{\sum_w \exp(u_w^T v_c)} \cdot \frac{\partial}{\partial v_c} \sum_w \exp(u_w^T v_c)$$

第二次链式法则（exp 的导数 × 内部函数的导数）：

$$\frac{\partial}{\partial v_c} \sum_w \exp(u_w^T v_c) = \sum_w \exp(u_w^T v_c) \cdot u_w$$

4.5.4 合并得到最终结果

将两项合并：

$$\frac{\partial}{\partial v_c} \log P(o | c) = u_o - \frac{\sum_{x=1}^V \exp(u_x^T v_c) \cdot u_x}{\sum_{w=1}^V \exp(u_w^T v_c)}$$

注意到分数部分恰好是 softmax 概率 $P(x | c)$ ，因此最终结果可以优雅地写为：

$$\frac{\partial}{\partial v_c} \log P(o | c) = u_o - \sum_{x=1}^V P(x | c) \cdot u_x$$

“观察值 - 期望值”的直觉

梯度具有非常直观的含义：

- u_o ： **观察到的**上下文词向量（实际出现的词）
- $\sum_x P(x | c) \cdot u_x$ ： **期望的**上下文词向量（模型预测的加权平均）

梯度 = 观察值 - 期望值。这意味着：

- 如果模型的预测完美匹配观察数据，梯度为零——已达到最优
- 否则，梯度会驱动参数调整，使模型的预测更接近实际观察

这种 “observed - expected” 的形式在统计学和机器学习中非常常见。

计算复杂度问题

在上述公式中，归一化项 $\sum_{w \in V} \exp(u_w^T v_c)$ 需要遍历整个词表（可能有几十万个词）。这使得每次梯度计算的复杂度为 $O(V)$ ，在实际中非常昂贵。后续课程将介绍**负采样**（negative sampling）等技巧来解决这个问题。

4.6 随机梯度下降 (SGD)

由于目标函数 $J(\theta)$ 是对语料库中**所有**窗口的求和（可能有数十亿个），计算完整梯度 $\nabla_{\theta} J(\theta)$ 的代价极其昂贵。

解决方案是**随机梯度下降** (Stochastic Gradient Descent, SGD)：

```
1 while True:
2     window = sample_window(corpus)
3     theta_grad = evaluate_gradient(J, window, theta)
4     theta = theta - alpha * theta_grad
```

Listing 1: SGD / Mini-batch Gradient Descent 伪代码

SGD 的核心思想

- 不计算完整梯度，而是每次**随机采样**一个（或一小批）窗口
- 基于这个小样本估计梯度并更新参数
- 虽然每次更新的方向有噪声，但**平均而言**会朝正确方向前进
- 实践中通常使用 **mini-batch**（小批量）而非单个样本，以兼顾效率和稳定性

4.7 本章小结

Word2Vec 的训练流程可以概括为：

1. **初始化**：为每个词随机生成中心向量和上下文向量
2. **遍历语料**：对每个位置，取中心词和窗口内的上下文词
3. **计算概率**：通过 softmax 计算上下文词的条件概率
4. **计算梯度**：利用“观察值 - 期望值”公式
5. **更新参数**：通过 SGD 调整词向量
6. **重复直到收敛**

最终得到的词向量能够编码丰富的语义信息，使得语义相似的词在向量空间中彼此接近。

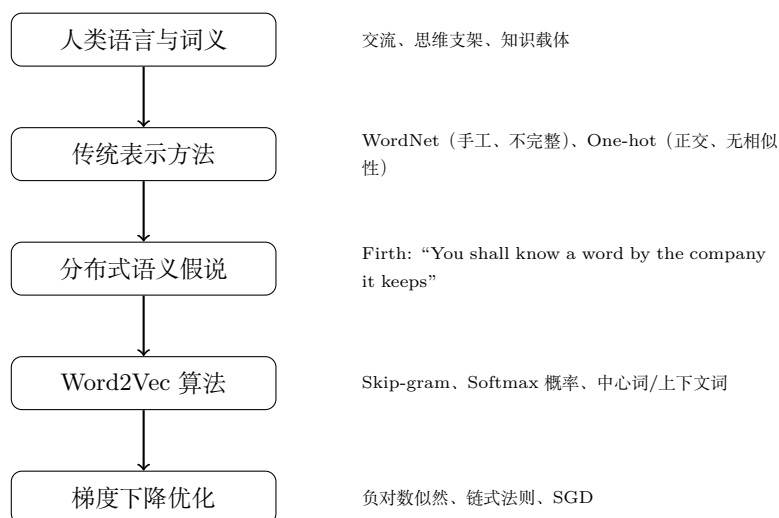
5 总结与延伸

5.1 讲者的核心总结

Manning 在课程结尾总结了本讲的核心线索：

1. 人类语言是最复杂的符号系统，也是 AI 最具挑战性的领域之一
2. 传统的词义表示（WordNet、one-hot）无法有效捕捉语义相似性
3. 分布式语义假说提供了全新的视角：“词义 = 上下文”
4. Word2Vec 通过简单的“预测上下文”任务，从大量文本中学习出高质量的词向量
5. 训练过程的核心是梯度下降——通过微积分指导参数调整方向

5.2 全课知识图谱



5.3 关键 Takeaways

五条核心要点

1. **语言是思维的支架**：人类语言不仅用于交流，更是高层次认知的基础工具
2. **分布式语义是词表示的正确范式**：词的含义由其上下文决定，而非人工标注的关系
3. **词向量 = 可计算的语义**：稠密低维向量使得语义相似性可以通过点积/余弦相似度直接度量
4. **Softmax 是万能的概率化工具**：将任意实数向量转化为概率分布，在深度学习中无处不在
5. **“观察值 - 期望值”驱动学习**：Word2Vec 的梯度具有直观的统计含义，通过不断缩小预测与观察之间的差距来学习

5.4 拓展阅读

- Mikolov et al., 2013. *Efficient Estimation of Word Representations in Vector Space*: <https://arxiv.org/abs/1301.3781> ——Word2Vec 原始论文
- Mikolov et al., 2013. *Distributed Representations of Words and Phrases and their Compositionality*: <https://arxiv.org/abs/1310.4546> ——负采样等改进

- Pennington et al., 2014. *GloVe: Global Vectors for Word Representation*: <https://nlp.stanford.edu/projects/glove/> ——另一种词向量方法
- CS224N 课程官网: <https://web.stanford.edu/class/cs224n/>
- Jay Alammr, *The Illustrated Word2Vec*: <https://jalammar.github.io/illustrated-word2vec/> ——优秀的可视化教程